

Reconhecimento de gestos de mão com redes neurais

Relatório de andamento — 21 de setembro de 2026

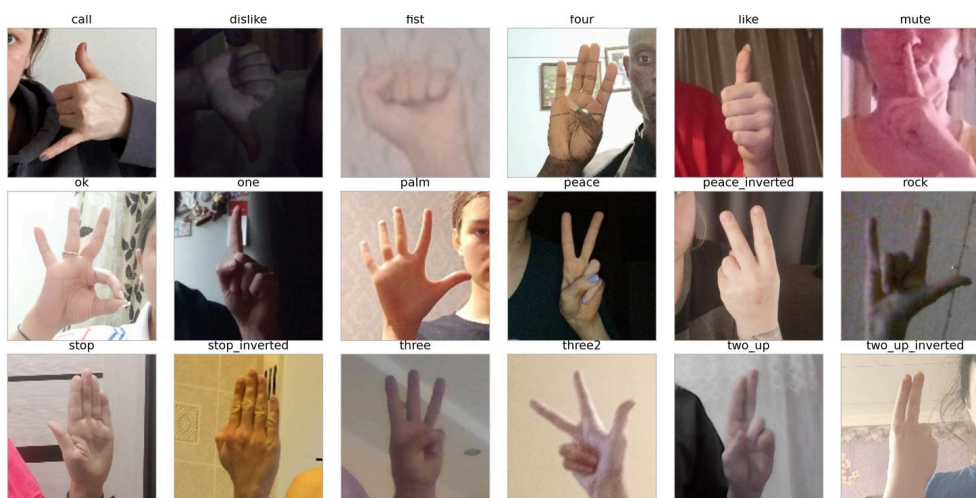
Resumo

- Ensinamos dois modelos de inteligência artificial a reconhecer **18 gestos de mão** em fotos. Os dois acertam **99,8%** das fotos de pessoas que nunca viram.
- Esse número foi conferido de cinco formas diferentes e **se sustenta**. Boa parte dos poucos erros que sobram são fotos que o próprio conjunto de dados rotulou errado.
- A **poda de 90%** dos pesos não aumenta os erros, e a **quantização int8** deixa o modelo até **8 vezes mais rápido**.
- A melhor combinação encontrada reconhece um gesto em **0,8 milésimo de segundo**, errando praticamente o mesmo que o modelo original.
- Com **um quarto das fotos de treino** o modelo ainda erra só 0,29%. Abaixo disso a qualidade cai rápido.
- As quatro etapas previstas estão concluídas. Falta a medição final de latência em GPU e as seeds 1 e 2 de um experimento extra de poda.

Como ler os números. Como os modelos quase não erram, comparar "99,79% com 99,80%" não diz nada. Por isso contamos **erros**: "1,5×" quer dizer "uma vez e meia os erros do modelo original". Cada experimento roda com 3 seeds (0, 1 e 2); "±" é a variação entre elas.

1. O problema e os dados

O modelo recebe a foto recortada em volta da mão e responde qual dos 18 gestos ela mostra. As fotos vêm do HaGRID, um conjunto público com pessoas do mundo todo fotografadas em casa.



	quantidade
Fotos usadas	278.715, de 20.117 pessoas
Para ensinar o modelo	175.591 fotos
Para acompanhar o aprendizado	19.511 fotos
Para a prova final	83.613 fotos, de 6.044 pessoas

Regra mais importante do projeto: **uma pessoa nunca aparece em dois grupos**. Se aparecesse, o modelo poderia reconhecer a pessoa ou o quarto dela, e não o gesto. A prova final só é usada uma vez por modelo, no fim.

2. Etapa 1 — dois modelos comparados

Testamos duas "arquiteturas" conhecidas, que organizam suas camadas de jeitos opostos. Ambas partem de um modelo já treinado em fotos gerais e são ajustadas para gestos.

	ResNet-50	DenseNet-121
Acurácia no teste	99,79%	99,80%
Erros no teste (de 83.613), seeds 0 / 1 / 2	177, 165, 178	160, 167, 167
Tamanho do modelo	90 MB	27 MB
Tempo de treino	1 h 26 min	1 h 47 min

Empate em qualidade. A diferença entre os dois é menor que a variação entre seeds do mesmo modelo. A DenseNet é três vezes menor, mas mais lenta.

3. O resultado é confiável?

Um acerto tão alto levanta suspeita, então fizemos uma auditoria.

Verificação	Resultado
Alguma pessoa em dois grupos?	Nenhuma
A lista de fotos confere com os arquivos originais?	278.715 de 278.715
Fotos repetidas entre treino e prova?	Nenhuma idêntica
Se embaralharmos as respostas certas no treino, o modelo ainda acerta?	Não: 4,9%, o mesmo que chutar
E em 278.702 fotos de 19.042 pessoas que ficaram de fora do projeto?	Acerta 99,77%

Um problema real foi encontrado no conjunto de dados: algumas pessoas têm mais de um cadastro, então 87 fotos da prova (0,1%) são quase iguais a fotos do treino. Retirando da prova todas as pessoas suspeitas, no pior cenário o acerto cai de 99,79% para 99,72%. Não muda a conclusão.

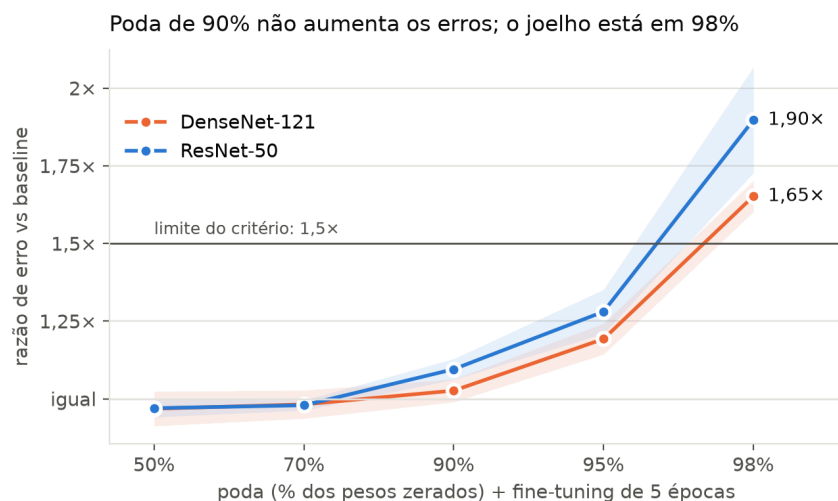
Por que sobram erros? Das cerca de 170 fotos erradas, 117 são erradas pelos **dois** modelos, e em 109 delas os dois dão a mesma resposta. Olhando essas fotos, muitas estão rotuladas errado na origem. Abaixo, seis exemplos: o rótulo do dataset e o que os dois modelos predizem.



4. Etapa 2 — deixar o modelo menor

Poda

Um modelo tem milhões de pesos, e muitos quase não influenciam a resposta. A poda zera os de menor valor; depois vem um fine-tuning de 5 épocas.



Poda	ResNet-50	DenseNet-121	Tamanho do arquivo
50% e 70%	igual	igual	61% e 42% do original
90%	1,10×	1,03×	21%
95%	1,28×	1,19×	cerca de 15%
98%	1,90×	1,65×	cerca de 11%

Nove em cada dez pesos são dispensáveis. **Limitação importante:** a poda deixa o arquivo menor, mas **não deixa o modelo mais rápido**, porque o computador faz as mesmas contas, só que com zeros.

Quantização int8

A segunda técnica guarda cada número do modelo em 8 bits em vez de 32. O arquivo fica com um quarto do tamanho e as contas ficam mais rápidas.

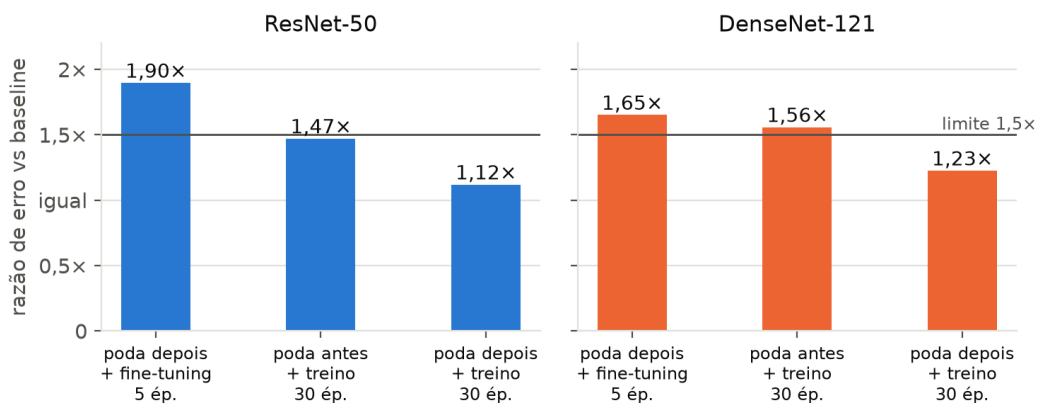
int8	ResNet-50	DenseNet-121
Erros, int8 em CPU (fbgemm)	1,04×	1,24×
Erros, int8 em GPU (TensorRT)	1,01×	1,07×
Tamanho	23 MB	8 MB

Para a ResNet a int8 sai de graça. A DenseNet perde qualidade em CPU.

Poda antes ou depois do treino?

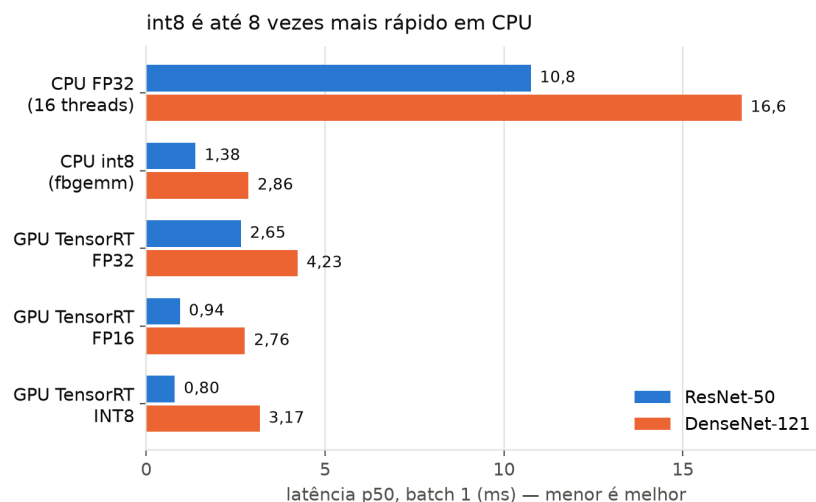
A pedido do orientador, testamos podar **antes** do treino. O resultado surpreendeu: a ordem importa pouco. **O que importa é quantas épocas o modelo treina depois da poda.**

Poda de 98%: o que decide é o número de épocas depois da poda



Com poda de 95%, as três formas dão resultados parecidos. Com 98%, treinar 30 épocas depois da poda resolve o problema nos dois modelos: a razão de erro cai de 1,90× para 1,12× na ResNet-50 e de 1,65× para 1,23× na DenseNet-121 (3 seeds). Podar antes do treino fica no meio. As seeds que faltam de "podar antes" estão rodando.

5. Etapa 3 — qual versão é a melhor?



Latência p50, batch 1 (ms)	ResNet-50	DenseNet-121
CPU, FP32	10,8	16,6
CPU, int8 (fbgemm)	1,38	2,86
GPU, TensorRT FP32	2,65	4,23
GPU, TensorRT INT8	0,80	3,17
Poda 90%, CPU FP32	10,9	16,5

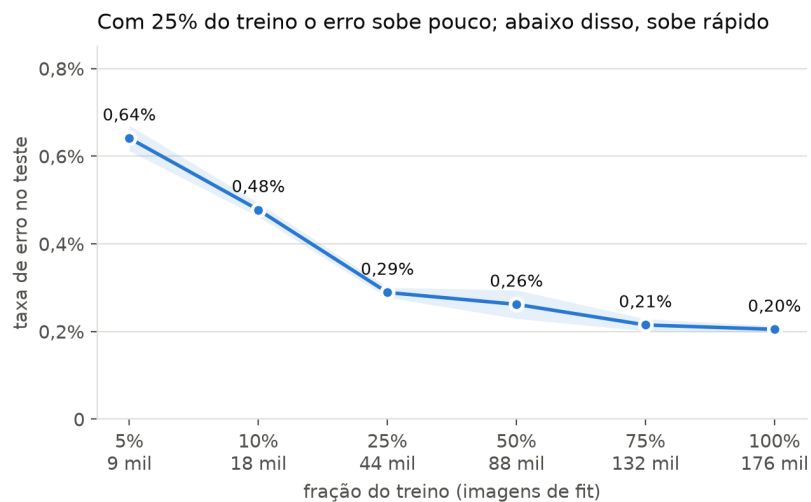
A regra de escolha foi definida **antes** de ver os resultados: entre as versões que erram no máximo 1,5× o original, vence a mais rápida na GPU.

Vencedora: ResNet-50 int8 TensorRT — 0,80 ms por foto, erros 1,01×, arquivo de 24 MB.

Três aprendizados: a DenseNet faz menos contas no papel, mas é mais lenta em todas as medições; a int8 da DenseNet ficou mais lenta que a FP16 no TensorRT; a poda não mudou a velocidade.

6. Etapa 4 — e se houvesse menos fotos?

Rotular fotos é caro. Treinamos a versão vencedora com cada vez menos fotos, sempre com a mesma prova final.



Fração do treino	Imagens	Erros (FP32)	Erros (TensorRT INT8)
100%	175.591	referência	referência
75%	131.693	1,05×	1,07×
50%	87.798	1,28×	1,31×
25%	43.898	1,41×	1,49×
10%	17.561	2,33×	2,45×
5%	8.782	3,13×	3,30×

Com um quarto das fotos, o erro sobe 41%. O ponto de virada fica entre 25% e 10%. Mesmo com 5% das fotos o modelo acerta 99,36%. A versão int8 acompanha a FP32 em todos os pontos.

7. Problemas encontrados e corrigidos

- **Treinos interrompidos cedo demais.** Uma regra de "parar quando deixar de melhorar" estava cortando 13 dos 18 treinos da etapa 4 antes da fase final, que é onde o modelo mais ganha. Um ponto chegou a 277 erros em vez de 165. A regra foi retirada e os treinos foram continuados de onde pararam.
- **Calibração da int8 instável no TensorRT.** A calibração usa uma amostra de 1.024 fotos. Com o método inicial, o mesmo modelo ia de 194 a 2.596 erros dependendo da amostra sorteada. Testamos quatro métodos e adotamos o mais estável, que varia de 62 a 64 erros onde o original faz 60. As 28 medições afetadas foram refeitas.
- **Memória estourada.** Esse mesmo ajuste chegou a usar 53 GB de memória e derrubou o computador duas vezes. Foi reescrito para trabalhar em partes pequenas.

8. Em andamento e pendente

Item	Situação
Seeds 1 e 2 da poda antes/depois do treino, em 95% e 98%	Rodando: 10 de 16 treinos concluídos, cerca de 10 horas restantes
Medição final de velocidade na placa de vídeo	Pendente: precisa ser feita com a tela gráfica fechada. Os tempos de placa de vídeo deste relatório são preliminares
Confirmar com o orientador dois ajustes feitos no caminho	Pendente: a retirada da parada antecipada e o critério de desempate quando duas versões têm praticamente a mesma velocidade
Artigo e pôster	A partir de 1º de novembro

9. O que já está entregue

- **Código, resultados e registros** de todos os treinos no repositório, com um comando que refaz todas as tabelas e gráficos a partir dos dados.
- **Modelos publicados** para download: os 6 modelos da etapa 1 e os 40 modelos comprimidos da etapa 2.
- **Aplicativo de demonstração** que usa a webcam: escolhe-se um dos modelos e ele reconhece o gesto feito dentro de um quadrado na tela.
- **Documentos:** metodologia, guia de reprodução, relatório da auditoria e análise dos erros.

Anexo — todos os acertos medidos

Teste: 83.613 imagens. Cada versão comprimida parte do baseline da mesma seed. "—" = não executado ou ainda rodando.

ResNet-50	Acurácia média	Seed 0	Seed 1	Seed 2	Erros (média)
Baseline	99,793%	99,788%	99,803%	99,787%	173
Poda 50% + fine-tuning 5 ép.	99,799%	99,801%	99,804%	99,792%	168
Poda 70% + fine-tuning 5 ép.	99,797%	99,791%	99,805%	99,795%	170
Poda 90% + fine-tuning 5 ép.	99,773%	99,773%	99,776%	99,770%	190
Poda 95% + fine-tuning 5 ép.	99,735%	99,725%	99,736%	99,744%	222
Poda 98% + fine-tuning 5 ép.	99,608%	99,614%	99,587%	99,622%	328
Poda 50% antes do treino (30 ép.)	99,790%	99,790%	—	—	176
Poda 70% antes do treino (30 ép.)	99,762%	99,762%	—	—	199
Poda 90% antes do treino (30 ép.)	99,772%	99,772%	—	—	191
Poda 95% antes do treino (30 ép.)	99,757%	99,757%	—	—	203
Poda 98% antes do treino (30 ép.)	99,700%	99,712%	99,688%	—	251
Poda 95% depois + treino 30 ép.	99,764%	99,751%	99,764%	99,775%	198

ResNet-50	Acurácia média	Seed 0	Seed 1	Seed 2	Erros (média)
Poda 98% depois + treino 30 ép.	99,768%	99,768%	99,778%	99,758%	194
int8 CPU (fbgemm)	99,785%	99,797%	99,792%	99,766%	180
TensorRT FP32	99,793%	99,790%	99,805%	99,786%	173
TensorRT FP16	99,792%	99,790%	99,801%	99,786%	174
TensorRT INT8	99,791%	99,793%	99,801%	99,780%	174
Poda 90% + int8 CPU	99,758%	99,758%	—	—	202
Poda 95% + int8 CPU	99,715%	99,715%	—	—	238
Poda 90% + TensorRT INT8	99,767%	99,767%	—	—	195
Poda 95% + TensorRT INT8	99,719%	99,719%	—	—	235

DenseNet-121	Acurácia média	Seed 0	Seed 1	Seed 2	Erros (média)
Baseline	99,803%	99,809%	99,800%	99,800%	165
Poda 50% + fine-tuning 5 ép.	99,809%	99,810%	99,799%	99,819%	159
Poda 70% + fine-tuning 5 ép.	99,807%	99,811%	99,795%	99,813%	162
Poda 90% + fine-tuning 5 ép.	99,798%	99,805%	99,787%	99,801%	169
Poda 95% + fine-tuning 5 ép.	99,765%	99,769%	99,754%	99,773%	196
Poda 98% + fine-tuning 5 ép.	99,675%	99,687%	99,659%	99,678%	272
Poda 50% antes do treino (30 ép.)	99,797%	99,797%	—	—	170
Poda 70% antes do treino (30 ép.)	99,795%	99,795%	—	—	171
Poda 90% antes do treino (30 ép.)	99,787%	99,787%	—	—	178
Poda 95% antes do treino (30 ép.)	99,758%	99,758%	—	—	202
Poda 98% antes do treino (30 ép.)	99,696%	99,685%	99,706%	—	254
Poda 95% depois + treino 30 ép.	99,783%	99,756%	99,798%	99,795%	181
Poda 98% depois + treino 30 ép.	99,759%	99,745%	99,763%	99,768%	202
int8 CPU (fbgemm)	99,756%	99,749%	99,757%	99,763%	204
TensorRT FP32	99,803%	99,809%	99,800%	99,799%	165
TensorRT FP16	99,803%	99,809%	99,801%	99,800%	164
TensorRT INT8	99,790%	99,791%	99,792%	99,788%	175
Poda 90% + int8 CPU	99,737%	99,737%	—	—	220
Poda 95% + int8 CPU	99,697%	99,697%	—	—	253
Poda 90% + TensorRT INT8	99,779%	99,779%	—	—	185
Poda 95% + TensorRT INT8	99,752%	99,752%	—	—	207

ResNet-50, eixo 4 (redução de dados)	Acurácia média	Seed 0	Seed 1	Seed 2	Erros (média)
100% do treino (175.591 imagens), FP32	99,795%	99,788%	99,794%	99,803%	171
75% do treino (131.693 imagens), FP32	99,785%	99,781%	99,774%	99,800%	180
50% do treino (87.798 imagens), FP32	99,738%	99,760%	99,701%	99,754%	219
25% do treino (43.898 imagens), FP32	99,711%	99,723%	99,700%	99,711%	242
10% do treino (17.561 imagens), FP32	99,522%	99,518%	99,540%	99,510%	399
5% do treino (8.782 imagens), FP32	99,359%	99,389%	99,331%	99,357%	536
100% do treino (175.591 imagens), TensorRT INT8	99,799%	99,793%	99,797%	99,809%	168
75% do treino (131.693 imagens), TensorRT INT8	99,785%	99,769%	99,782%	99,804%	180
50% do treino (87.798 imagens), TensorRT INT8	99,738%	99,764%	99,696%	99,752%	219
25% do treino (43.898 imagens), TensorRT INT8	99,703%	99,714%	99,691%	99,702%	249
10% do treino (17.561 imagens), TensorRT INT8	99,510%	99,501%	99,524%	99,505%	410
5% do treino (8.782 imagens), TensorRT INT8	99,340%	99,367%	99,315%	99,337%	552